Simulating Failure Modes and Defenses

systemdesignschool.io/fundamentals/cache-lab-disasters

Lab: The Disaster Drill

The previous article described three failure modes in theory. This lab simulates each one so you can observe the database impact and verify that defenses work.

Exercise 1: Cache Stampede and Request Coalescing

Simulate a hot key expiring while 100 concurrent readers try to access it. First without protection (all hit the database), then with a lock that coalesces requests.

Stampede: Without vs With Locking

Loading Python environment... This may take a few seconds on first load.

Without the lock, all 100 threads race to the database simultaneously. With request coalescing, only the first thread queries the database. The other 99 wait for the result and read from the freshly-populated cache.

Exercise 2: Cache Penetration and Bloom Filter

Simulate an attacker requesting random non-existent keys. First without protection (all requests hit the database), then with a Bloom filter that blocks impossible keys.

Penetration: Bloom Filter Defense

Loading Python environment... This may take a few seconds on first load.

The Bloom filter catches most non-existent keys without any database interaction. A small percentage may pass through as false positives (the Bloom filter says "might exist" when it doesn't), but the database load drops from 500 queries to near zero.

Exercise 3: Cache Avalanche and TTL Jitter

Simulate 1000 entries all expiring at the same time versus staggered expiration with jitter.

Avalanche: Synchronized vs Jittered Expiration

Loading Python environment... This may take a few seconds on first load.

Without jitter, all 1000 entries expire in the same instant—the database receives a burst of 1000 queries at once. With jitter, expirations spread across a 12-second window. The peak load drops from 1000 to roughly 80–100 queries per second—an order of magnitude reduction that the database can handle comfortably.

These three exercises demonstrate the failure modes and their defenses. In a system design interview, being able to name the failure mode, explain why it happens, and describe the fix shows depth beyond basic cache usage.